
Chapter Three

Syntax Analysis

Compiled By: Kassahun T.

Chapter Three : Syntax Analysis

- Parser (Syntax Analysis)
- The Role of the Parser
- Predictive Parsing
- Operator Precedence Parsing
- LR Parsers
- Syntax Error Handling
- Regular Expressions Vs. Context Free Grammars
- Left Recursion (Reading Assignment)
- Derivation Trees (Reading Assignment)

Chapter Three : Syntax Analysis

Syntax Analyzer

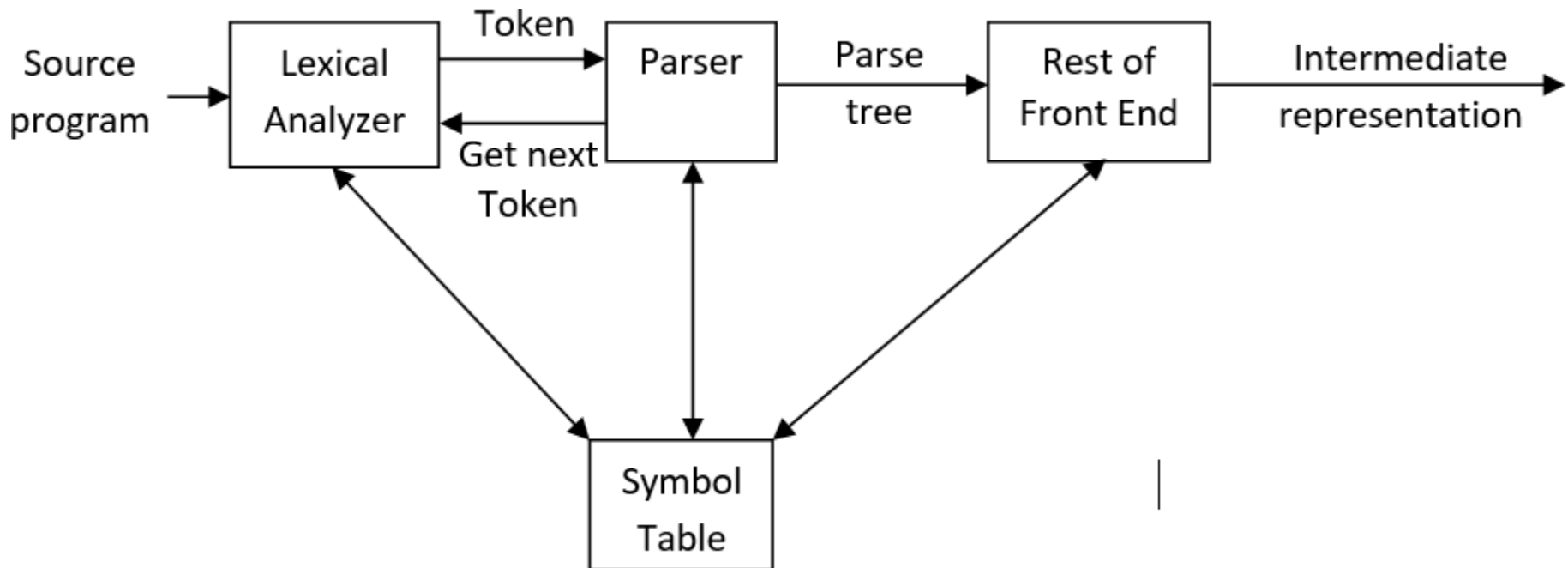
- By design every programming language has **precise rules** that prescribe the syntactic structure of well-formed programs.
- *Syntax Analyzer* creates the syntactic structure of the given source program.
- The syntax of a programming is described by a *context-free grammar (CFG)*. We will use BNF (Backus-Naur Form) Or CNF notation in the description of CFGs.
- This syntactic structure is mostly a *parse tree*. Syntax Analyzer is also known as *parser*.

Chapter Three : Syntax Analysis

- The syntax analyzer (parser) checks whether a given source program satisfies the rules implied by a context-free grammar or not.
 - If it satisfies, the parser creates the parse tree of that program.
 - Otherwise the parser gives the error messages.

Chapter Three : Syntax Analysis

- Parser works on a stream of tokens.
- The smallest item is a token (in Syntax Analyzer).



Chapter Three : Syntax Analysis

- We categorize the parsers into two groups:
 - **Top-Down Parser** : the parse tree is created top to bottom, starting from the root.
 - **Bottom-Up Parser** : the parse is created bottom to top; starting from the leaves.
- Both top-down and bottom-up parsers scan the input from left to right (**one symbol/token at a time**).
- Efficient top-down and bottom-up parsers can be implemented only for sub-classes of context-free grammars.
 - LL for top-down parsing
 - LR for bottom-up parsing

Chapter Three : Syntax Analysis

The Role of a Parser

- In our compiler model, the parser obtains a string of tokens from the lexical analyzer, and verifies that the string of token names can be generated by the source language.
- It is expected that the *parser reports any syntax errors* in an intelligible fashion and to recover from commonly occurring errors to continue processing the remainder of the program.

Chapter Three : Syntax Analysis

The Role of a Parser

- Conceptually, for well-formed programs, the parser constructs a parse tree and passes it to the rest of the compiler for further processing,
- The methods commonly used in compilers can be classified as being either top-down or bottom-up.

Chapter Three : Syntax Analysis

Syntax Error Handling (Reading Assignment)

- The error handler should be written with the following goals in mind:
 - Errors should be reported clearly and accurately.
 - The compiler should recover and detect other errors.
 - It should not slow down the whole process (of compilation) significantly.

Chapter Three : Syntax Analysis

Syntax Error Handling (Reading Assignment)

- Fortunately most errors are straight forward and easy to detect.

E.g. of common errors:

missing ; missing : in :=

- The error handler should report:
 - the place in the source program where the error is detected.
 - the type of error (if possible)

Chapter Three : Syntax Analysis

Syntax Error Handling (Reading Assignment)

○ There are four main strategies in error handling:

- **Panic mode recovery** : discards all tokens until a synchronization token is found. `int a, 5abcd, sum, $2;`

- **Phrase level recovery**: the parser makes a local correction so that it can continue to parse the rest of the input. `id + + num ;`

-deleting +
-inserting ident.
- Substitute

- **Error productions**: augment the grammar to capture the most common errors that programmers make.

`stmt → expr ; | expr {Error: Missing
semicolon at end of statement}`

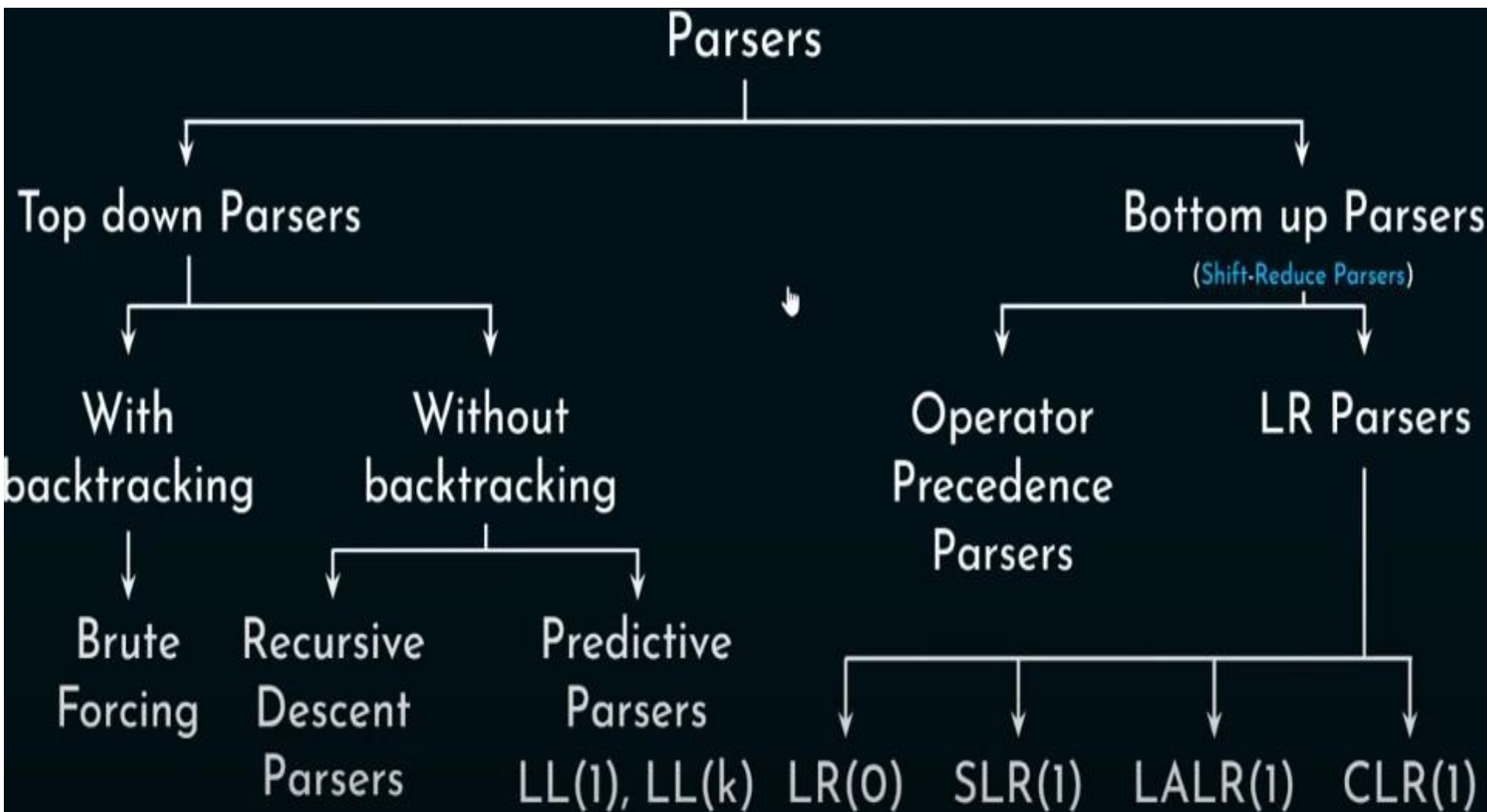
- **Global correction**: makes as few changes as possible in the program so that a globally least cost correction program is obtained.

Chapter Three : Type of Parsing

Parser

- **Parser** is that phase of compiler which takes token string as input and with the help of existing grammar, converts it into the corresponding parse tree.
- Parser is also known as Syntax Analyzer.

Chapter Three : Type of Parsing



Chapter Three : Type of Parsing

Top-Down Parsing

- Top-down parsing can be viewed as the problem of constructing a parse tree for the input string, starting from the root and creating the nodes of the parse tree in preorder (depth-first) .
- Equivalently, top-down parsing can be viewed as finding a leftmost derivation for an input string.

Chapter Three : Type of Parsing

Top-Down Parsing

- Top-down parser is the parser which generates parse for the given input string with the help of grammar productions by expanding the non-terminals i.e. it starts from the start symbol and ends on the terminals.
- Further Top-down parser is classified into 2 types: full backtracking parser, and Non- backtracking parser.

Chapter Three : Type of Parsing

NOTE

- In order to construct Top down parser the CFG should not have:
 - **Left – Recursion** - Left recursion in a grammar can cause an infinite loop in a top-down parser, as it may continually apply the same production rule without progressing in the input.
 - **Non – Determinism** - Non-determinism occurs when a grammar has multiple possible ways to parse an input sequence, leading to ambiguity in choosing the correct production rule.

Chapter Three : Top-Down Parsing

With Backtracking Parsing

- This method of top-down parsing can be considered as an attempt to find the **left most derivation** for an input string .
- Which may require backtracking to find the correct A-production to be applied.
- Therefore, it is also known as **Brute force parser** or the with backtracking parser.
- It basically generates the parse tree by using brute force or backtracking. It may involve **backtracking** (make repeated scans of the input and the production).

Chapter Three : Type of Parsing

Example TDP

The sequence of parse trees for the input $\text{id} + \text{id} * \text{id}$

With production given. $E \rightarrow E + E \mid E * E \mid \text{id}$

Chapter Three : Top-Down Parsing

Recursive Descent Parsing:

- Recursive Descent Parser uses the technique of Top-Down Parsing without backtracking.
- It can be defined as a Parser that uses the various recursive procedure to process the input string with no backtracking.
- It can be simply performed using a Recursive language.
- The major approach of recursive-descent parsing is to relate each non-terminal with a procedure.

Chapter Three : Top-Down Parsing

Recursive Descent Parsing: Example

○Consider the following grammar having rules:

$$\begin{aligned} E &\rightarrow iE' \\ E' &\rightarrow +iE' \mid \varepsilon \end{aligned}$$

```
E()  
{  
    if(look_ahead=='i')  
    {  
        match('i');  
        E'();  
    }  
}
```

```
E'()  
{  
    if(look_ahead=='+')  
    {  
        match('+');  
        match('i');  
        E'();  
    }  
    else  
        return;  
}
```

```
match(char c)  
{  
    if(look_ahead==c)  
        look_ahead = getchar();  
    else  
        printf("ERROR!");  
}
```

```
main()  
{  
    E();  
    if(look_ahead=='$')  
        printf("Parsing Successful!");  
}
```

Chapter Three : Top-Down Parsing

Recursive Descent Parsing: Example

$$E \rightarrow iE'$$
$$E' \rightarrow +iE' \mid \varepsilon$$

○ Consider the following grammar having rules:

```
1. E()
2. {
3.     if(look_ahead='i')
4.     {
5.         match('i');
6.         E'();
7.     }
8. }
```

```
1. E'()
2. {
3.     if(look_ahead='+')
4.     {
5.         match('+');
6.         match('i');
7.         E'();
8.     }
9.     else
10.         return;
11. }
```

```
1. match(char c)
2. {
3.     if(look_ahead=c)
4.         look_ahead = getchar();
5.     else
6.         printf("ERROR!");
7. }
```

```
1. main()
2. {
3.     E();
4.     if(look_ahead='$')
5.         printf("Parsing Successful!");
6. }
```



Input: i + i \$

Chapter Three : Top-Down Parsing

Recursive Descent Parsing: To construct the parse tree using RDP:

- Create a one node tree consisting of **S**.
- Two pointers, one for the **tree** and one for the **input**, will be used to indicate where the parsing process is ?
- Initially, they will be **on S** and the **first input symbol**, respectively.
- Then we use the first S-production to expand the tree. The tree pointer will be positioned on the **left most symbol (nonterminal)** of the newly created sub-tree.

Chapter Three: Top-Down Parsing

Recursive Descent Parsing: To construct the parse tree using RDP:

- *As the symbol pointed by the tree pointer matches that of the symbol pointed by the input pointer, both pointers are moved to the right.*
- Whenever the tree pointer points on a **non-terminal**, we expand it using the first production of the non terminal.

Chapter Three : Top-Down Parsing

Non-recursive descent parser (Predictive or LL(1)):

- Predictive parsing chooses the correct A-production by **looking ahead** at the input a fixed number of symbols , typically we may look only at one (that is, the next input symbol) .

Chapter Three : Top-Down Parsing

Non-recursive descent parser (Predictive Parser or LL(1)):

- It is also known as LL(1) parser or **predictive parser** or **without backtracking parser** or it is **dynamic parser**. It uses **parsing table** to generate the parse tree instead of backtracking.
- It is possible to build a non-recursive parser by explicitly maintaining a stack.
- This method uses a **parsing table** that determines the next **production to be applied**.

Chapter Three : Top-Down Parsing

Non-recursive descent parser (Predictive or LL(1)):

- Here the 1st **L** represents that the scanning of the Input will be done from **Left to Right manner** and second **L** shows that in this Parsing technique we are going to use **Left most Derivation** Tree. and finally the **1** represents the number of look ahead, means how many symbols are you going to see when you want to make a decision.

Chapter Three : Top-Down Parsing

FIRST and FOLLOW

- The construction of both top-down and bottom-up parsers is aided by two functions, *FIRST and FOLLOW*, associated with a grammar *G*.
- During top down parsing, *FIRST and FOLLOW* allow us to choose which production to apply, based on the next input symbol.

Chapter Three : Top-Down Parsing

FIRST

○ $\text{FIRST}(X)$ for a grammar symbol X is the set of terminals that begin the strings derivable from X .

○ Rules to compute FIRST Set:

1. If x is a terminal, then $\text{FIRST}(x) = \{ 'x' \}$
2. If $x \rightarrow \epsilon$, is a production rule, then add ϵ to $\text{FIRST}(x)$.
3. If $X \rightarrow Y_1 Y_2 Y_3 \dots Y_n$ is a production,
 1. $\text{FIRST}(X) = \text{FIRST}(Y_1)$
 2. If $\text{FIRST}(Y_1)$ contains ϵ then $\text{FIRST}(X) = \{ \text{FIRST}(Y_1) - \epsilon \} \cup \{ \text{FIRST}(Y_2) \}$
 3. If $\text{FIRST}(Y_i)$ contains ϵ for all $i = 1$ to n , then add ϵ to $\text{FIRST}(X)$.

Chapter Three : Top-Down Parsing

Example

1. Production Rules of Grammar(Find First)

$E \rightarrow TE'$

$E' \rightarrow +T E' \mid \epsilon$

$T \rightarrow F T'$

$T' \rightarrow *F T' \mid \epsilon$

$F \rightarrow (E) \mid id$

2. Production Rules of Grammar (Find First)

$S \rightarrow ACB \mid Cbb \mid Ba$

$A \rightarrow da \mid BC$

$B \rightarrow g \mid \epsilon$

$C \rightarrow h \mid \epsilon$

Chapter Three : Top-Down Parsing

Example

3. Production Rules of Grammar (Find First)

$$\begin{array}{lcl} S & \rightarrow & Bb \mid cd \\ B & \rightarrow & aB \mid \epsilon \\ C & \rightarrow & cC \mid \epsilon \end{array}$$

4. Production Rules of Grammar (Find First)

$$\begin{array}{lcl} S & \rightarrow & ABCDE \\ A. & \rightarrow & a \mid \epsilon \\ B. & \rightarrow & b \mid \epsilon \\ C. & \rightarrow & c \\ D. & \rightarrow & d \mid \epsilon \\ E. & \rightarrow & e \mid \epsilon \end{array}$$

Chapter Three : Top-Down Parsing

FOLLOW

- **Follow(X)** to be the set of terminals that can appear immediately to the right of Non-Terminal X in some sentential form.
- Rules to compute **Follow** Set:

- 1) $\text{FOLLOW}(S) = \{ \$ \}$ // where S is the starting Non-Terminal
- 2) If $A \rightarrow pBq$ is a production, where p, B and q are any grammar symbols, then everything in $\text{FIRST}(q)$ except ϵ is in $\text{FOLLOW}(B)$.
- 3) If $A \rightarrow pB$ is a production, then everything in $\text{FOLLOW}(A)$ is in $\text{FOLLOW}(B)$.
- 4) If $A \rightarrow pBq$ is a production and $\text{FIRST}(q)$ contains ϵ , then $\text{FOLLOW}(B)$ contains $\{ \text{FIRST}(q) - \epsilon \} \cup \text{FOLLOW}(A)$

Chapter Three : Top-Down Parsing

NOTE

- ϵ may appear in the first function of a non-terminal.
- ϵ will never appear in the follow function of a non-terminal.
- Before calculating the first and follow functions, eliminate Left Recursion from the grammar, if present.

Chapter Three : Top-Down Parsing

Example

1. Production Rules of Grammar(Find Follow)

$E \rightarrow TE'$
 $E' \rightarrow +T E' \mid \epsilon$
 $T \rightarrow F T'$
 $T' \rightarrow *F T' \mid \epsilon$
 $F \rightarrow (E) \mid id$

2. Production Rules of Grammar (Find Follow)

$S \rightarrow ACB \mid Cbb \mid Ba$
 $A \rightarrow da \mid BC$
 $B \rightarrow g \mid \epsilon$
 $C \rightarrow h \mid \epsilon$

Chapter Three : Top-Down Parsing

Example

3. Production Rules of Grammar (Find Follow)

$$\begin{array}{l} S \rightarrow Bb \mid cd \\ B \rightarrow aB \mid \epsilon \\ C \rightarrow cC \mid \epsilon \end{array}$$

4. Production Rules of Grammar (Find Follow)

$$\begin{array}{l} S \rightarrow ABCDE \\ A \rightarrow a \mid \epsilon \\ B \rightarrow b \mid \epsilon \\ C \rightarrow c \\ D \rightarrow d \mid \epsilon \\ E \rightarrow e \mid \epsilon \end{array}$$

Chapter Three : Top-Down Parsing

Production Rules:

$E \rightarrow TE'$

$E' \rightarrow +T E' \mid \epsilon$

$T \rightarrow F T'$

$T' \rightarrow *F T' \mid \epsilon$

$F \rightarrow (E) \mid id$

FIRST set

$FIRST(E) = FIRST(T) = \{ (, id \}$

$FIRST(E') = \{ +, \epsilon \}$

$FIRST(T) = FIRST(F) = \{ (, id \}$

$FIRST(T') = \{ *, \epsilon \}$

$FIRST(F) = \{ (, id \}$

FOLLOW Set

$FOLLOW(E) = \{ \$,) \}$ // Note ')' is there because of 5th rule

$FOLLOW(E') = FOLLOW(E) = \{ \$,) \}$ // See 1st production rule

$FOLLOW(T) = \{ FIRST(E') - \epsilon \} \cup FOLLOW(E') \cup FOLLOW(E) = \{ +, \$,) \}$

$FOLLOW(T') = FOLLOW(T) = \{ +, \$,) \}$

$FOLLOW(F) = \{ FIRST(T') - \epsilon \} \cup FOLLOW(T') \cup FOLLOW(T) = \{ *, +, \$,) \}$

Chapter Three: Top-Down Parsing

Production Rules:

$S \rightarrow ACB \mid Cbb \mid Ba$

$A \rightarrow da \mid BC$

$B \rightarrow g \mid \epsilon$

$C \rightarrow h \mid \epsilon$

FIRST set

$\text{FIRST}(S) = \text{FIRST}(A) \cup \text{FIRST}(B) \cup \text{FIRST}(C) = \{ d, g, h, \epsilon, b, a \}$

$\text{FIRST}(A) = \{ d \} \cup \text{FIRST}(B) = \{ d, g, \epsilon \}$

$\text{FIRST}(B) = \{ g, \epsilon \}$

$\text{FIRST}(C) = \{ h, \epsilon \}$

FOLLOW Set

$\text{FOLLOW}(S) = \{ \$ \}$

$\text{FOLLOW}(A) = \{ h, g, \$ \}$

$\text{FOLLOW}(B) = \{ a, \$, h, g \}$

$\text{FOLLOW}(C) = \{ b, g, \$, h \}$

Chapter Three : Top-Down Parsing

Construction of a predictive parsing table

$E \rightarrow TE'$
 $E' \rightarrow +T E' \mid \epsilon$
 $T \rightarrow F T'$
 $T' \rightarrow *F T' \mid \epsilon$
 $F \rightarrow (E) \mid id$

- Now, after computing the First and Follow set for each *Non-Terminal symbol* we have to construct the Parsing table.
- In the table Rows will contain the Non-Terminals and the column will contain the Terminal Symbols.
- All the **Null Productions** of the Grammars will go under the Follow elements and the remaining productions will lie under the elements of First set.(True /False)

Chapter Three : Top-Down Parsing

Construction of a predictive parsing or LL(1) table

$E \rightarrow TE'$

$E' \rightarrow +T E' \mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *F T' \mid \epsilon$

$F \rightarrow (E) \mid id$

FIRST set

$FIRST(E) = FIRST(T) = \{ (, id \}$

$FIRST(E') = \{ +, \epsilon \}$

$FIRST(T) = FIRST(F) = \{ (, id \}$

$FIRST(T') = \{ *, \epsilon \}$

$FIRST(F) = \{ (, id \}$

FOLLOW Set

$FOLLOW(E) = \{ \$,) \}$ // Note ')' is there because of 5th rule

$FOLLOW(E') = FOLLOW(E) = \{ \$,) \}$ // See 1st production rule

$FOLLOW(T) = \{ FIRST(E') - \epsilon \} \cup FOLLOW(E') \cup FOLLOW(E) = \{ +, \$,) \}$

$FOLLOW(T') = FOLLOW(T) = \{ +, \$,) \}$

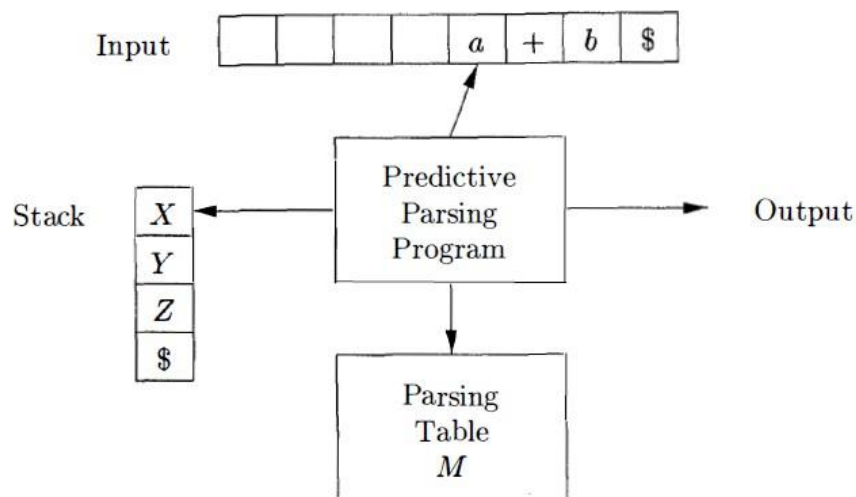
$FOLLOW(F) = \{ FIRST(T') - \epsilon \} \cup FOLLOW(T') \cup FOLLOW(T) = \{ *, +, \$,) \}$

NON - TERMINAL	INPUT SYMBOL					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

Chapter Three : Top-Down Parsing

Non-recursive Predictive Parsing or LL(1)

- A non-recursive predictive parser can be built by maintaining a stack **explicitly**, rather implicitly via recursive calls(inputs).
- This method, shown in Figure, uses a parsing table that determines the next production to be applied.



Chapter Three : Top-Down Parsing

Non-recursive Predictive Parsing

- **Buffer** contains the string to be parsed followed by \$ (the right end marker)
- The **stack** contains a sequence of grammar symbols with \$ at the bottom. Initially, the stack contains the start symbol of the grammar followed by \$.
- The parsing table is a two dimensional array $M[X, a]$ where X is a nonterminal of the grammar and a is a terminal or \$

Chapter Three : Top-Down Parsing

Non-recursive Predictive Parsing shows moves made by a predictive parser on input **id + id * id**

E . -> **TE'**

E' -> **+T E' | ε**

T -> **FT'**

T' -> ***F T' | ε**

F . -> **(E) | id**

MATCHED	STACK	INPUT	ACTION
	<i>E</i> \$	id + id * id\$	
	<i>TE'</i> \$	id + id * id\$	output <i>E</i> → <i>TE'</i>
	<i>FT'E'</i> \$	id + id * id\$	output <i>T</i> → <i>FT'</i>
	id <i>T'E'</i> \$	id + id * id\$	output <i>F</i> → id
id	<i>T'E'</i> \$	+ id * id\$	match id
id	<i>E'</i> \$	+ id * id\$	output <i>T'</i> → ε
id	+ <i>TE'</i> \$	+ id * id\$	output <i>E'</i> → + <i>TE'</i>
id +	<i>TE'</i> \$	id * id\$	match +
id +	<i>FT'E'</i> \$	id * id\$	output <i>T</i> → <i>FT'</i>
id +	id <i>T'E'</i> \$	id * id\$	output <i>F</i> → id
id + id	<i>T'E'</i> \$	* id\$	match id
id + id	* <i>FT'E'</i> \$	* id\$	output <i>T'</i> → * <i>FT'</i>
id + id *	<i>FT'E'</i> \$	id\$	match *
id + id *	id <i>T'E'</i> \$	id\$	output <i>F</i> → id
id + id * id	<i>T'E'</i> \$	\$	match id
id + id * id	<i>E'</i> \$	\$	output <i>T'</i> → ε
id + id * id	\$	\$	output <i>E'</i> → ε

NON - TERMINAL	INPUT SYMBOL					
	id	+	*	(	)	\$
<i>E</i>	<i>E</i> → <i>TE'</i>			<i>E</i> → <i>TE'</i>		
<i>E'</i>		<i>E'</i> → + <i>TE'</i>			<i>E'</i> → ε	<i>E'</i> → ε
<i>T</i>	<i>T</i> → <i>FT'</i>			<i>T</i> → <i>FT'</i>		
<i>T'</i>		<i>T'</i> → ε	<i>T'</i> → * <i>FT'</i>		<i>T'</i> → ε	<i>T'</i> → ε
<i>F</i>	<i>F</i> → id			<i>F</i> → (<i>E</i>)		

Chapter Three : Bottom-Up Parsing

- Bottom-up parsing can be defined as an attempt to reduce the input string **W** to the start symbol of grammar by tracing out the rightmost derivations of **W** in reverse.
- Bottom-up parsing starts from the **leaf nodes of a tree** and works in upward direction till it reaches the root node.
- Here, we start from a sentence and then apply production rules in reverse manner in order to reach the start symbol.

Chapter Three : Bottom-Up Parsing

- In the bottom up parsing, the parsing starts with the input symbol and construct the parse tree up to the start symbol by tracing out the rightmost derivations of string in reverse.
- Bottom up parsing is also known as shift-reduce parsing.
- Bottom-up parsers don't need left factored grammars.

Chapter Three : Bottom-Up Parsing

- A bottom-up parse corresponds to the construction of a parse tree for an input string beginning at the leaves (the bottom) and working up towards the root (the top) .

$E \rightarrow T$

$T \rightarrow T * F$

$T \rightarrow \text{id}$

$F \rightarrow T$

$F \rightarrow \text{id}$

- Example Given production and input **id * id**

$F * \text{id}$
|
 id

$T * \text{id}$
|
 F
|
 id

$T * F$
| |
 F id
|
 id

T
/ | \
 T $*$ F
| |
 F id
|
 id

E
|
 T
/ | \
 T $*$ F
| |
 F id
|
 id

Chapter Three : Bottom-Up Parsing

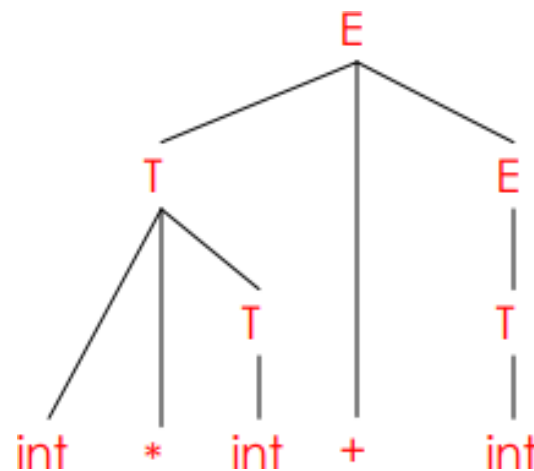
Example 2

Given Production: $E \rightarrow T + E \mid T$ $T \rightarrow \text{int} * T \mid \text{int} \mid (E)$

- Consider the string: $\text{int} * \text{int} + \text{int}$
- Bottom-up parsing reduces a string to the start symbol by inverting productions:

$\text{int} * \text{int} + \text{int}$
 $\text{int} * T + \text{int}$
 $T + \text{int}$
 $T + T$
 $T + E$
 E

$T \rightarrow \text{int}$
 $T \rightarrow \text{int} * T$
 $T \rightarrow \text{int}$
 $E \rightarrow T$
 $E \rightarrow T + E$



Chapter Three : Bottom-Up Parsing

- Bottom-up parsing uses two kinds of actions: **Shift, Reduce**
- Shift reduce parsing is a process of **reducing a string to the start symbol of a grammar.**
- Shift reduce parsing uses a **stack to hold the grammar** and an **input tape (buffer) to hold the string.**
- At the **shift action**, the current symbol in the input string is **pushed to a stack.**
- At each **reduction**, the symbols will be replaced by the non-terminals (The top of the stack). The symbol is the right side of the production and non-terminal is the left side of the production.

A String $\xrightarrow{\text{reduce to}}$ the starting symbol

Chapter Three : Bottom-Up Parsing

- Bottom-up parsing uses two kinds of actions: **Shift, Reduce**
- $E \rightarrow T + E \mid T$ $T \rightarrow \text{int} * T \mid \text{int} \mid (E)$
- Consider the string: $\text{int} * \text{int} + \text{int}$

int * int + int	shift
int * int + int	shift
int * int + int	shift
int * int + int	reduce $T \rightarrow \text{int}$
int * T + int	reduce $T \rightarrow \text{int} * T$
T + int	shift
T + int	shift
T + int	reduce $T \rightarrow \text{int}$
T + T	reduce $E \rightarrow T$
T + E	reduce $E \rightarrow T + E$
E	

Chapter Three : Bottom-Up Parsing

○ Example Given production (Below) input **id * id**

$E \rightarrow T$

$T \rightarrow T * F$

$T \rightarrow \text{id}$

$T \rightarrow F$

$F \rightarrow \text{id}$

$F \rightarrow \text{id}$

$T \rightarrow F \rightarrow \text{id}$

$T \rightarrow T * F \rightarrow \text{id} * \text{id}$

Partial parse tree for $T \rightarrow T * F$

Complete parse tree for $E \rightarrow T$

STACK	INPUT	ACTION
\$	id ₁ * id ₂ \$	shift
\$id ₁	* id ₂ \$	reduce by $F \rightarrow \text{id}$
\$F	* id ₂ \$	reduce by $T \rightarrow F$
\$T	* id ₂ \$	shift
\$T *	id ₂ \$	shift
\$T * id ₂	\$	reduce by $F \rightarrow \text{id}$
\$T * F	\$	reduce by $T \rightarrow T * F$
\$T	\$	reduce by $E \rightarrow T$
\$E	\$	accept

Chapter Three : Bottom-Up Parsing

N.B: While the primary operations are **shift and reduce**, there are actually four possible actions a shift-reduce parser can make:

(1) shift, (2) reduce, (3) accept, and (4) error.

- **Shift** : Shift the next input symbol onto the top of the stack.
(pushing)
- **Reduce** : The right end of the string to be reduced at the top of the stack. Locate the left end of the string within the stack and decide with what nonterminal to replace the string.
- **Accept** : Announce successful completion of parsing.
- **Error** : Discover a syntax error and call an error recovery routine.

Chapter Three: Bottom-Up Parsing

○ Example Given production (Below) input **a1-(a2+a3)**

$S \rightarrow S+S$

$S \rightarrow S-S$

$S \rightarrow (S)$

$S \rightarrow a$

Stack contents	Input string	Actions
\$	a1-(a2+a3)\$	shift a1
\$a1	-(a2+a3)\$	reduce by $S \rightarrow a$
\$S	-(a2+a3)\$	shift -
\$S-	(a2+a3)\$	shift (
\$S-(	a2+a3)\$	shift a2
\$S-(a2	+a3)\$	reduce by $S \rightarrow a$
\$S-(S	+a3) \$	shift +
\$S-(S+	a3) \$	shift a3
\$S-(S+a3	) \$	reduce by $S \rightarrow a$
\$S-(S+S	) \$	shift)
\$S-(S+S)	\$	reduce by $S \rightarrow S+S$
\$S-(S)	\$	reduce by $S \rightarrow (S)$
\$S-S	\$	reduce by $S \rightarrow S-S$
\$S	\$	Accept

Chapter Three : LR Parsers

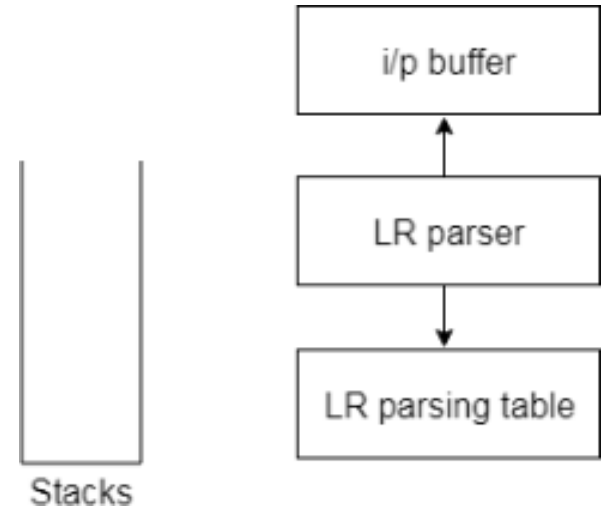
LR(K) Parser

- LR parsing is one type of bottom up parsing. It is used to parse the **large class of grammars**.
- In the LR parsing, "**L**" stands for **left-to-right** scanning of the input.
- "**R**" stands for constructing **a right most derivation** in reverse.
- "**K**" is the number of **input symbols** of the **look-ahead** used to make number of parsing decision.
- LR parsing is divided into four parts: **LR(0) parsing**, **SLR(1) parsing**, LALR(1) parsing and CLR(1) parsing, The first 2 use LR item 0 , and the second 2 use LR item 1.

Chapter Three : LR Parsers

LR algorithm:

- All LR algorithm requires four major things these are : **stack, input, output and parsing table.**
- **N.B:** In all type of LR parsing, **input, output and stack are same but parsing table is different.**
- **Input buffer:** is used to indicate end of input, and it contains the string to be parsed followed by a \$ Symbol.
- **A stack:** is used to contain a sequence of grammar symbols with a \$ at the bottom of the stack.
- Parsing table is a two dimensional array. It contains two parts: **Action part** and **Go To part.**
 - An LR parser makes **shift-reduce decisions** by maintaining states to keep track of where we are in a parse.
 - We need two functions : **Closure()** and **Goto()**



Chapter Three : LR Parsers

Steps which are involving in LR parsing

1. For the given input write context free grammar
2. Check the ambiguity of the grammar
3. Add augment production of the given grammar
4. Create canonical collection of LR(0) items.
5. Draw the data flow diagram (DFD)
6. Contract LR parsing table

Chapter Three : LR Parsers

LR(0) Parser

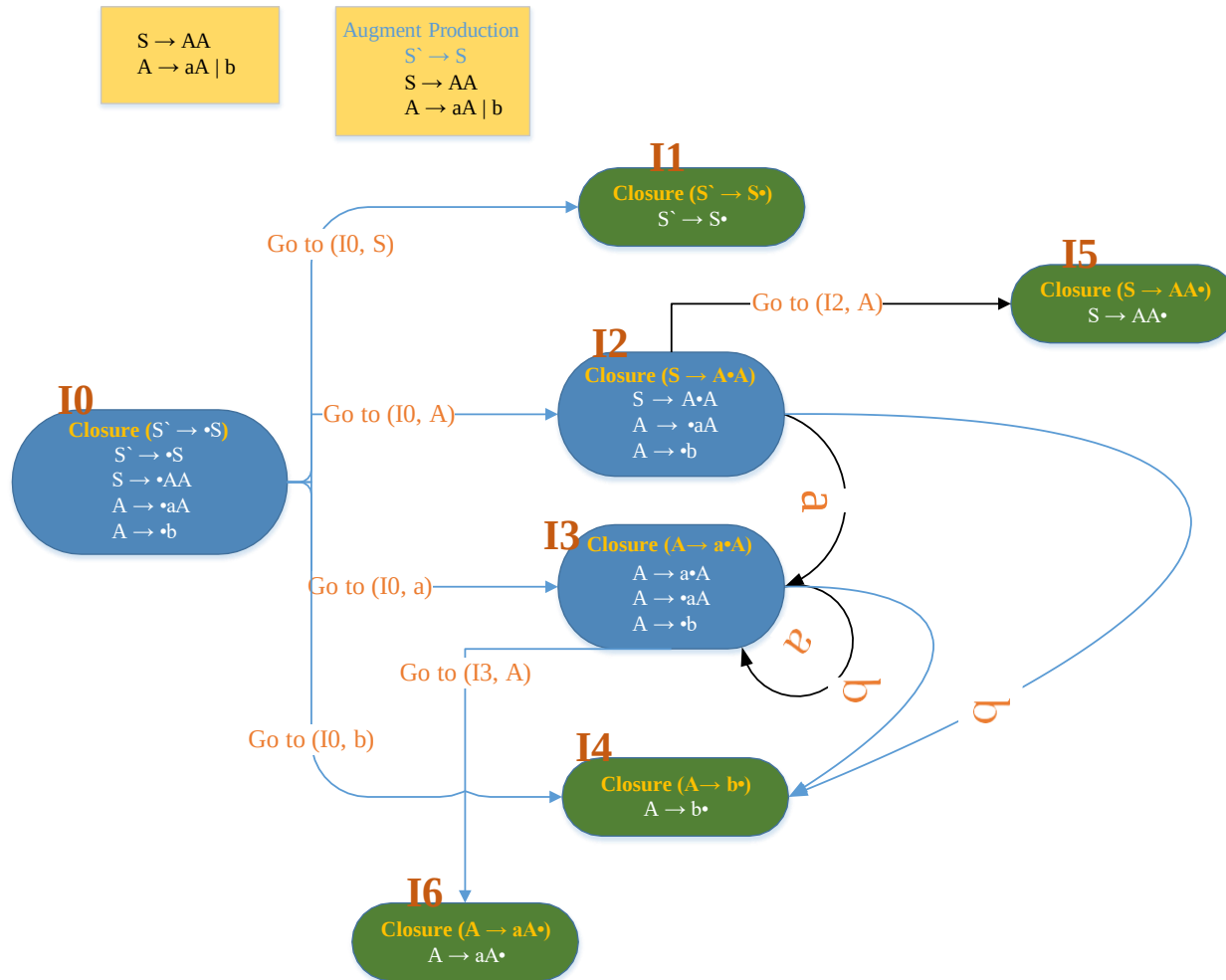
Canonical Collection of LR(0) items

- **An LR (0) item** is a production **G** with dot at some position on the right side of the production.
- LR(0) **items** is useful to indicate that **how much of the** input has been scanned up to a given point in the process of parsing.
- **In the LR (0), we place the reduce node in the entire row.**
- States represent sets of items, An LR(0) item of a grammar G is a production of G with a dot at some position of the body. Thus, production $A \rightarrow XYZ$ yields the four items.
- The production $A \rightarrow \epsilon$ generates only one item $A \rightarrow \cdot \epsilon$

$$\begin{aligned} A &\rightarrow \cdot XYZ \\ A &\rightarrow X \cdot YZ \\ A &\rightarrow XY \cdot Z \\ A &\rightarrow XYZ \cdot \end{aligned}$$

Chapter Three : LR(0) Parsers

This is Canonical Collection of LR(0) items

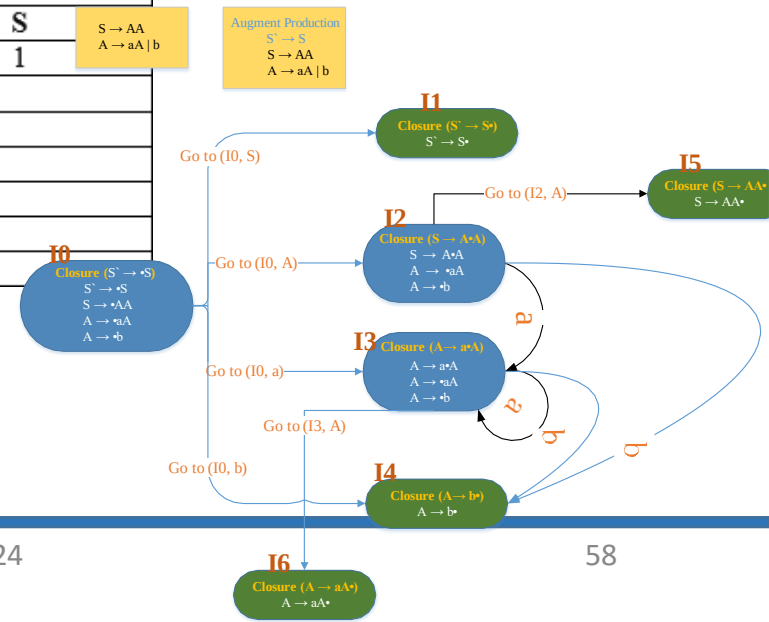


Chapter Three : LR(0) Parsers

LR(0) Table

- If a state is going to some other state on a terminal then it correspond to a **shift move. Or Reduce(if the state is final state).**
- If a state is going to some other state on a variable(non-terminal) then it correspond to **go to move.**
- If a state contain the **final item** in the particular row then write the reduce node completely.

States	Action			Go to	
	a	b	S	A	S
I ₀	S3	S4		2	1
I ₁			accept		
I ₂	S3	S4		5	
I ₃	S3	S4		6	
I ₄	r3	r3	r3		
I ₅	r1	r1	r1		
I ₆	r2	r2	r2		



Chapter Three: Bottom-Up Parsing

LR(0) Table

Explanation:

- I0 on S is going to I1 so write it as 1.
- I0 on A is going to I2 so write it as 2.
- I2 on A is going to I5 so write it as 5.
- I3 on A is going to I6 so write it as 6.
- I0, I2 and I3 on a are going to I3 so write it as S3 which means that shift 3.
- I0, I2 and I3 on b are going to I4 so write it as S4 which means that shift 4.
- I4, I5 and I6 all states contains the final item because they contain • in the right most end. So rate the production as production number.

Chapter Three : LR(0) Parsers

LR(0) Table

Based on the above parser pars input “aabb”

States	Action			Go to	
	a	b	\$	A	S
I ₀	S3	S4		2	1
I ₁			accept		
I ₂	S3	S4		5	
I ₃	S3	S4		6	
I ₄	r3	r3	r3		
I ₅	r1	r1	r1		
I ₆	r2	r2	r2		

STACK	SYMBOLS	INPUT	ACTION
0	\$	aabb\$	Shift to 3
0 3	\$a	abb\$	Shift to 3
0 3 3	\$aa	bb\$	Shift to 3
0 3 3 4	\$aab	b\$	Reduced to 3 Reduced By A->b
0 3 3	\$aaA	b\$	GoTo(A,3)
0 3 3-6	\$aaA	b\$	Reduced to 2 Reduced By A->aA
0 3	\$aA	b\$	GoTo(A,3)
0 3-6	\$aA	b\$	Reduced to 2 Reduced By A->aA
0	\$A	b\$	GoTo(A,0)
0 2	\$A	b\$	Shift 4
0 2 4	\$Ab	\$	Reduced to 3 Reduced By A->b
0 2	\$AA	\$	GoTo(A,2)
0 2-5	\$AA	\$	Reduced to 1 Reduced By S->AA
0	\$S	\$	GoTo(S,0)
0 1	\$S	\$	Accepted

Chapter Three: SLR(1) Parsers

SLR (1) Parsing

- **SLR (1)** refers to **simple LR** Parsing. It is same as LR(0) parsing. The only difference is in the **parsing table**.
- To construct SLR (1) parsing table, we use **canonical collection of LR (0) item**.
- **In the SLR (1) parsing, we place the reduce move only in the follow of left hand side.**
- Various steps involved in the SLR (1) Parsing:
 - For the given input string write a context free grammar
 - Check the ambiguity of the grammar
 - Add Augment production in the given grammar
 - Create Canonical collection of LR (0) items
 - Draw a data flow diagram (DFA)
 - Construct a SLR (1) parsing table

Chapter Three: SLR(1) Parsers

SLR (1) Parsing

- SLR (1) refers to simple LR Parsing. It is same as LR(0) parsing. The

SLR (1) Grammar

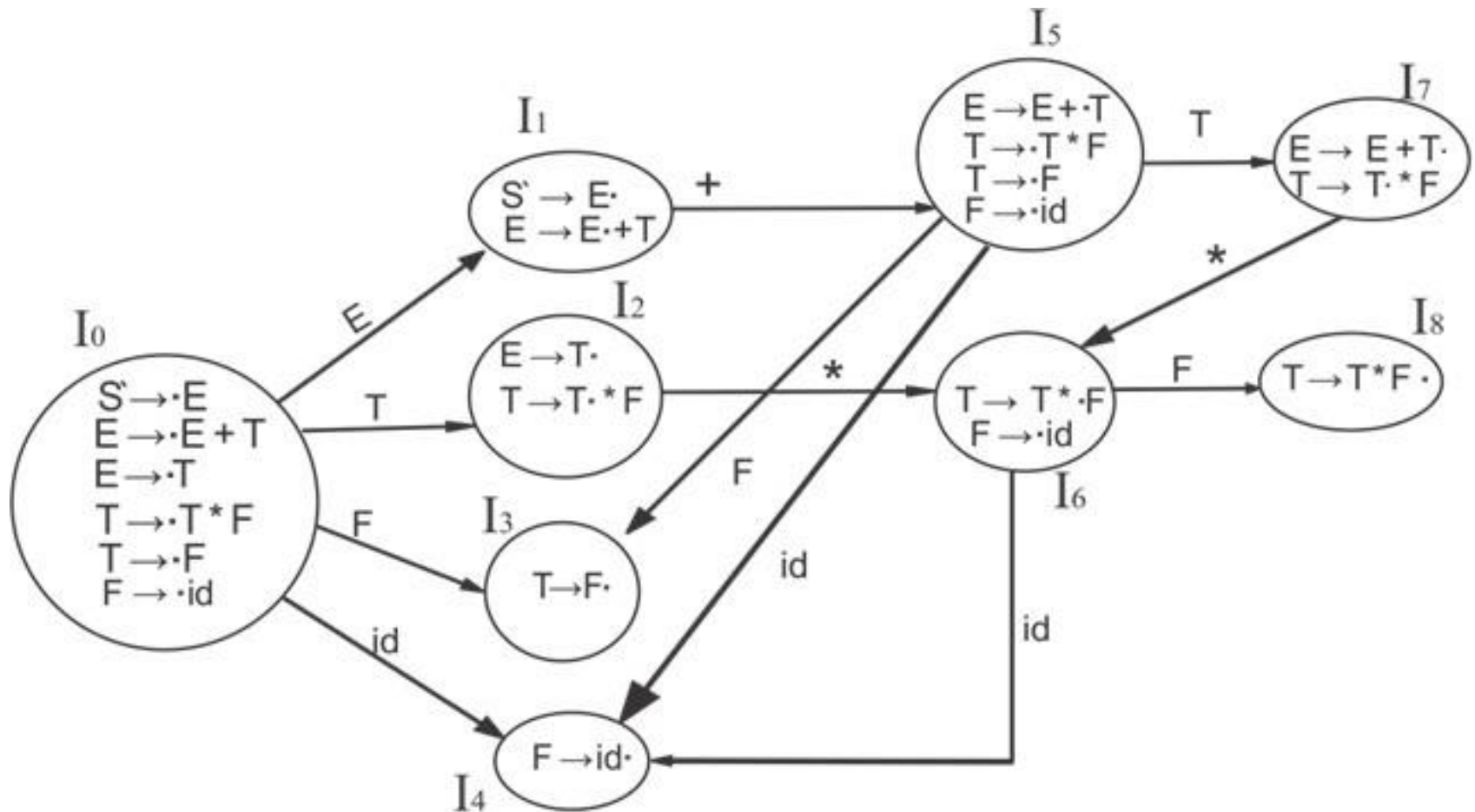
$$\begin{aligned} S &\rightarrow E \\ E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow \text{id} \end{aligned}$$

- Add Augment Production and insert ' $\bullet$ ' symbol at the first position for every production in G

$$\begin{aligned} S' &\rightarrow \bullet E \\ E &\rightarrow \bullet E + T \\ E &\rightarrow \bullet T \\ T &\rightarrow \bullet T * F \\ T &\rightarrow \bullet F \\ F &\rightarrow \bullet \text{id} \end{aligned}$$

Chapter Three : SLR(1) Parsers

SLR (1) Parsing



Chapter Three: SLR(1) Parsers

SLR (1) Parsing

Explanation:

$$\text{First (E)} = \text{First (E + T)} \cup \text{First (T)}$$

$$\text{First (T)} = \text{First (T * F)} \cup \text{First (F)}$$

$$\text{First (F)} = \{\text{id}\}$$

$$\text{First (T)} = \{\text{id}\}$$

$$\text{First (E)} = \{\text{id}\}$$

$$\text{Follow (E)} = \text{First (+T)} \cup \{\$\} = \{+, \$\}$$

$$\begin{aligned} \text{Follow (T)} &= \text{First (*F)} \cup \text{First (F)} \\ &= \{*, +, \$\} \end{aligned}$$

$$\text{Follow (F)} = \{*, +, \$\}$$

Chapter Three : SLR(1) Parsers

SLR (1) Parsing

- I1 contains the final item which drives $S \rightarrow E\bullet$ and follow (S) = { $\$$ }, so action {I1, $\$$ } = Accept
- I2 contains the final item which drives $E \rightarrow T\bullet$ and follow (E) = {+, $\$$ }, so action {I2, +} = R2, action {I2, $\$$ } = R2
- I3 contains the final item which drives $T \rightarrow F\bullet$ and follow (T) = {+, *, $\$$ }, so action {I3, +} = R4, action {I3, *} = R4, action {I3, $\$$ } = R4
- I4 contains the final item which drives $F \rightarrow id\bullet$ and follow (F) = {+, *, $\$$ }, so action {I4, +} = R5, action {I4, *} = R5, action {I4, $\$$ } = R5
- I7 contains the final item which drives $E \rightarrow E + T\bullet$ and follow (E) = {+, $\$$ }, so action {I7, +} = R1, action {I7, $\$$ } = R1
- I8 contains the final item which drives $T \rightarrow T * F\bullet$ and follow (T) = {+, *, $\$$ }, so action {I8, +} = R3, action {I8, *} = R3, action {I8, $\$$ } = R3.

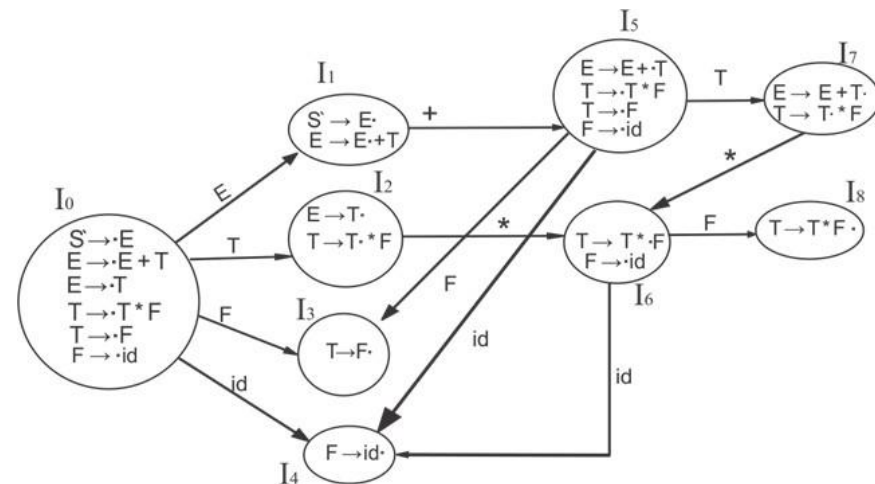
Chapter Three : SLR(1) Parsers

SLR (1) Parsing

Follow (E) = First (+T) $\cup$ {\$} = {+, \$}

Follow (T) = First (*F) $\cup$ First (F)
= {*, +, \$}

Follow (F) = {*, +, \$}



States	Action				Go to		
	id	+	*	\$	E	T	F
I ₀	S ₄				1	2	3
I ₁		S ₅		Accept			
I ₂		R ₂	S ₆	R ₂			
I ₃		R ₄	R ₄	R ₄			
I ₄		R ₅	R ₅	R ₅			
I ₅	S ₄					7	3
I ₆	S ₄						8
I ₇		R ₁	S ₆	R ₁			
I ₈		R ₃	R ₃	R ₃			

Chapter Three : CLR(1) Parsing

CLR (1) Parsing

- CLR refers to **canonical look-ahead**. CLR parsing use the canonical collection of LR (1) items to build the CLR (1) parsing table.
- CLR (1) parsing table produces the more number of states as compare to the SLR (1) parsing.
- In the CLR (1), we place the reduce node only in the look-ahead symbols.

Chapter Three: CLR(1) Parsing

CLR (1) Parsing

- LR (1) item is a collection of LR (0) items and a look ahead symbol.
- **LR (1) item = LR (0) item + look ahead**
- The look ahead is used to determine that where we place the final item. The look ahead always add \$ symbol for the argument production.
- **In the CLR (1), we place the reduce node only in the look-ahead symbols.**
- Various steps involved in the CLR (1) Parsing:
 - For the given input string write a context free grammar
 - Check the ambiguity of the grammar
 - Add Augment production in the given grammar
 - Create Canonical collection of LR (0) items
 - Draw a data flow diagram (DFA)
 - Construct a CLR (1) parsing table

Chapter Three : CLR(1) Parsing

CLR (1) Parsing

Example

$S \rightarrow AA$

$A \rightarrow aA$

$A \rightarrow b$

Add Augment Production, insert ' $\bullet$ ' symbol at the first position for every production in G and also add the look-ahead.

$S' \rightarrow \bullet S, \$$

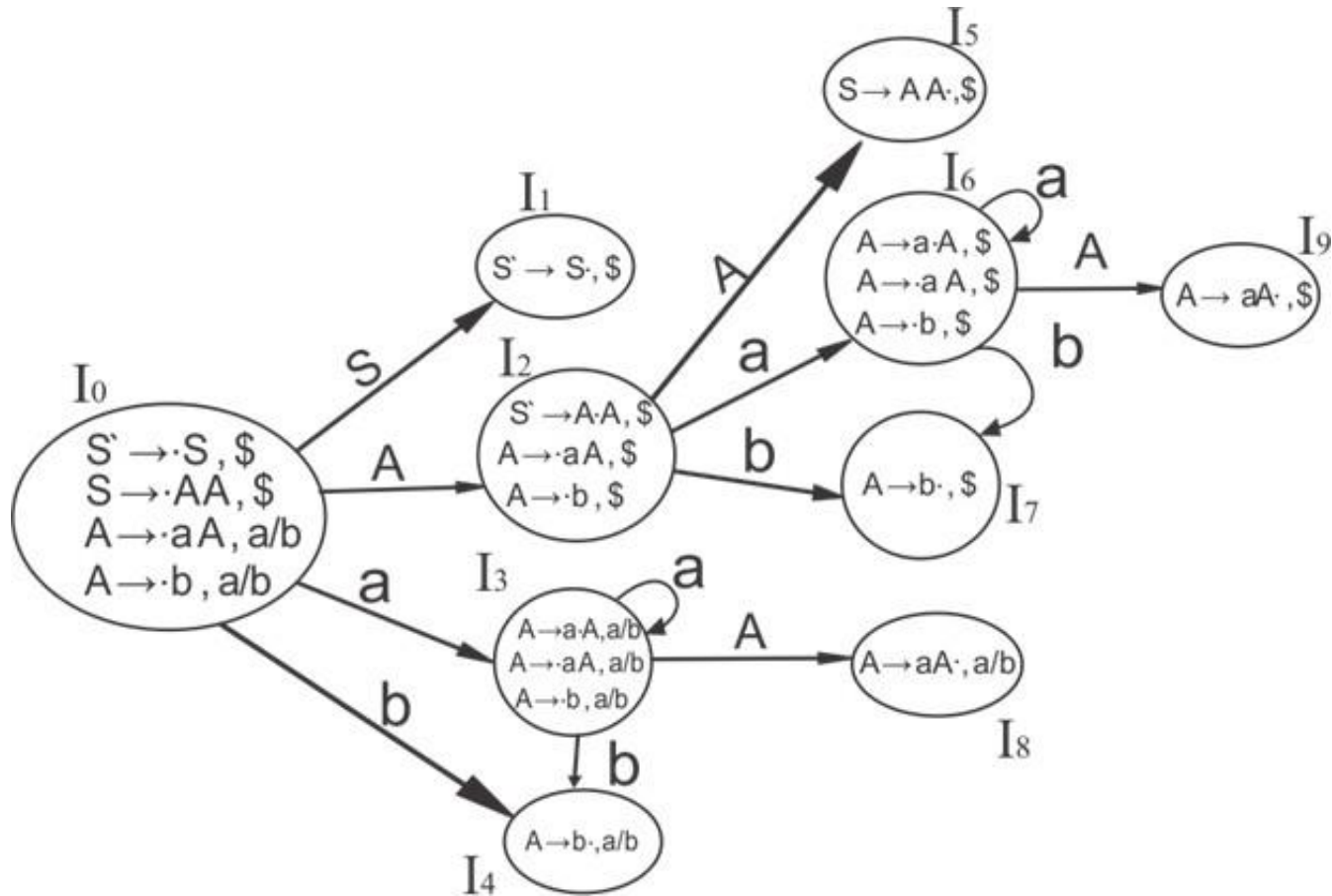
$S \rightarrow \bullet AA, \$$

$A \rightarrow \bullet aA, a/b$

$A \rightarrow \bullet b, a/b$

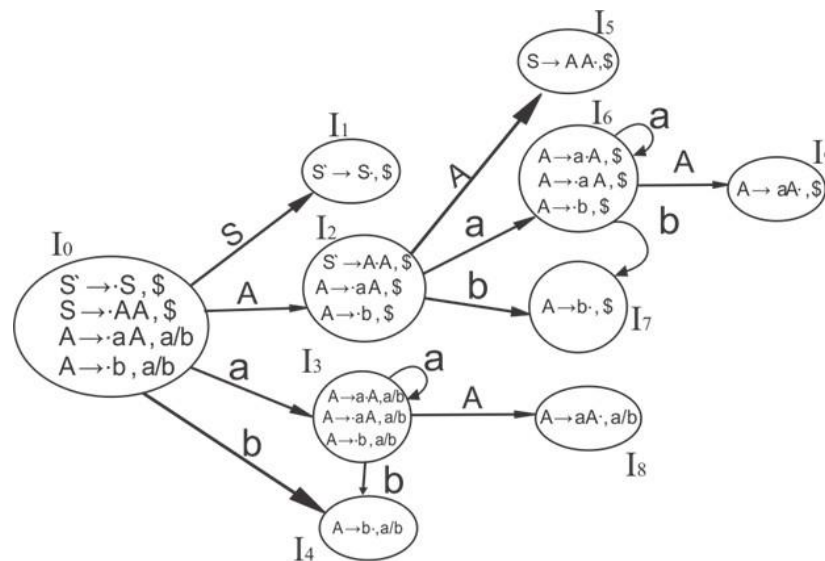
Chapter Three: Bottom-Up Parsing

CLR (1) Parsing DFA



Chapter Three: Bottom-Up Parsing

CLR (1) Parsing Table



States	a	b	S	S	A
I_0	S_3	S_4			2
I_1	Accept				
I_2	S_6	S_7			5
I_3	S_3	S_4			8
I_4	R_3	R_3			
I_5	R_1				
I_6	S_6	S_7			9
I_7	R_3				
I_8	R_2	R_2			
I_9	R_2				

Chapter Three : Bottom-Up Parsing

LALR (1) Parsing:

- LALR refers to the **look-ahead** LR. To construct the LALR (1) parsing table, we use the canonical collection of LR (1) items.
- In the LALR (1) parsing, the LR (1) items which have same productions but different look ahead are combined to form a single set of items.
- LALR (1) parsing is same as the CLR (1) parsing, only difference in the parsing table.

Use

<https://www.javatpoint.com/clr-1-parsing>

Chapter Three : Syntax Analysis

Thank You !!!!